

What is Monte Carlo Simulation?

Monte Carlo simulation is a technique that uses **random numbers** to model and study systems that involve uncertainty or randomness. Instead of solving a problem analytically (with exact equations), you simulate many possible outcomes by generating random inputs and observing the results.

The name comes from the famous **Monte Carlo Casino** in Monaco — a place built on randomness and probability. The technique was formally developed in the 1940s by mathematicians **Stanislaw Ulam** and **John von Neumann** while working on nuclear weapon design, where they needed to simulate random neutron behaviour.

Core idea: If a real system has random behaviour that is hard to analyse mathematically, you can *imitate* that randomness on a computer using random numbers, run the simulation many times, and collect statistics on what happens.

The Role of Random Numbers

A **random number** is a value generated in a way that gives no predictable pattern — every value in the range is equally likely. In Monte Carlo simulation, random numbers are used to **simulate randomness** in real-world systems.

More precisely, in simulation we typically work with **random numbers between 0 and 1** (uniform on $[0, 1]$). These are then mapped to whatever distribution the real system follows.

Random numbers are used to simulate uncertainty in:

- **Customer arrivals** (when will the next customer show up?)
- **Machine failures** (when will the machine break down?)
- **Inventory demand** (how many units will be demanded today?)
- **Service times** (how long will it take to serve this customer?)

In every simulation involving chance, random numbers are the engine. Each random number in a sequence represents one "draw from nature" — one random outcome.

Example: In a bank queue simulation, random numbers simulate the random arrival times of customers. In an inventory simulation, they determine the daily demand. Each random

number in the sequence drives one event.

How Monte Carlo Simulation Works

The process follows a clear pattern:

Step 1: Identify the Probability Distribution

Based on historical data or experience, determine how likely each outcome is (e.g., how often demand is 0, 15, 25, 35, etc.).

Step 2: Build a Cumulative Probability Table

Convert the probabilities into **cumulative probabilities** (each row = probability of being *at or below* that value).

Step 3: Assign Random Number Ranges

Divide the interval [0.00, 1.00] into segments proportional to the probability of each outcome. Each outcome gets a **random number range**.

Step 4: Generate (or Use) Random Numbers

Use a sequence of random numbers — either from a random number table, a calculator, or a computer.

Step 5: Map Each Random Number to an Outcome

For each random number, find which range it falls into — that range determines the simulated outcome.

Step 6: Analyse the Results

Compute statistics (mean, totals, shortfalls, etc.) from the simulated outcomes.

Example 1: Shopping Mall Customer Arrivals

The following table shows customer arrival patterns in a shopping mall. Random numbers are used to simulate how many customers arrive during each time period.

Random Number Range	Number of Customers
0.00 – 0.29	1 customer
0.30 – 0.69	3 customers
0.70 – 0.79	(from image — partially visible; likely 5 customers)
0.80 – 1.00	(likely 7 customers)

How to read the table: If a random number falls between 0.00 and 0.29, that period has 1 customer arrival. If it falls between 0.30 and 0.69, there are 3 arrivals. And so on. The wider the range, the more likely that outcome.

Simulation

Given random number sequence: **0.15, 0.82, 0.45, 0.76, 0.38**

Simulate the arrivals of customers using this random number sequence.

Period	Random Number	Range It Falls In	Simulated Arrivals
1	0.15	0.00 – 0.29	1 customer
2	0.82	0.80 – 1.00	7 customers
3	0.45	0.30 – 0.69	3 customers
4	0.76	0.70 – 0.79	5 customers
5	0.38	0.30 – 0.69	3 customers

Total simulated arrivals over 5 periods = 1 + 7 + 3 + 5 + 3 = 19 customers

Average arrivals per period = 19 / 5 = 3.8 customers

Interpretation: Even though the table only has four possible outcomes, the random number sequence produces a realistic mix of busy and quiet periods — just as a real shopping mall would experience.

Example 2: Inventory Simulation — Baker's Daily Demand

A baker keeps stock of a popular bread brand. Daily demand is based on past experience. Random numbers are used to predict daily demand for planning purposes.

Step 1: Demand Distribution Table

Daily Demand	Probability	Cumulative Probability	Random Number Range
0	0.01	0.01	0.00 – 0.00
15	0.15	0.16	0.01 – 0.15
25	0.20	0.36	0.16 – 0.35
35	0.50	0.86	0.36 – 0.85
45	0.12	0.98	0.86 – 0.97
50	0.02	1.00	0.98 – 0.99

How to read the table: 50% of the time, demand is 35 loaves — so numbers 0.36 to 0.85 all map to demand = 35. Only 2% of the time is demand 50, so only numbers 0.98–0.99 map to it.

Step 2: Random Number Sequence

Given sequence: **0.48, 0.78, 0.09, 0.51, 0.56, 0.77, 0.15, 0.14, 0.68, 0.09**

Step 3: Simulate 10 Days of Demand

Map each random number to the demand range it falls in:

Day	Random Number	Simulated Demand
1	0.48	35
2	0.78	35*
3	0.09	15
4	0.51	35
5	0.56	35
6	0.77	35
7	0.15	15
8	0.14	15
9	0.68	35
10	0.09	15

Note: The class notes record Day 2 (0.78) as demand = 45. Both 35 and 45 are possible depending on the exact RN range boundaries used. Using the table above (0.86–0.97 for 45), 0.78 falls in 0.36–0.85, giving demand = 35. If your lecturer uses slightly different ranges, always follow your lecturer's table.

Step 4: Estimate Daily Average Demand

$$\text{Average demand} = \frac{35 + 35 + 15 + 35 + 35 + 35 + 15 + 15 + 35 + 15}{10} = \frac{270}{10} = 27 \text{ loaves/day}$$

(Your notes record this as $270/10 = 27$ loaves per day.)

Example 3: Stock Situation — Baker's Inventory Analysis

Using the demand simulation above, now suppose the baker decides to **bake exactly 35 loaves per day** for 10 days. The question is: how many loaves remain (or are in deficit) at the end of each day?

Formula

$$\text{Stock Remaining} = \text{Stock Produced} - \text{Demand Simulated}$$

If the result is **positive** → surplus (leftover loaves). If **negative** → shortfall (unmet demand, lost sales or backlog).

Stock Table

Day	Produced	Simulated Demand	Stock Remaining	Interpretation
1	35	35	0	Exactly met
2	35	45	−10	Shortfall of 10
3	35	0	+35	35 left over
4	35	35	0	Exactly met
5	35	35	0	Exactly met
6	35	45	−10	Shortfall of 10
7	35	0	+35	35 left over
8	35	35	0	Exactly met
9	35	45	−10	Shortfall of 10
10	35	0	+35	35 left over

(Demand values here match your class notes' day-by-day stock results. Minor differences may exist depending on the RN boundary used for day 2, 6, 9.)

Analysis

- **Days with shortfall:** Days 2, 6, 9 — shortfall of 10 loaves each = **30 loaves unmet** across 10 days.
- **Days with surplus:** Days 3, 7, 10 — 35 loaves leftover each = **105 loaves wasted** (or carried over) across 10 days.
- **Days exactly met:** Days 1, 4, 5, 8.

What Should the Baker Do?

This simulation reveals a key business insight: producing a fixed 35 loaves per day is **not optimal** when demand varies. Options include:

1. **Produce more** (e.g., 40 or 45 loaves) to cover shortfall days — but this increases waste on low-demand days.
2. **Carry over surplus** from one day to the next (if bread doesn't go stale).
3. **Produce to the simulated average** (27 loaves) — but this causes shortfalls on higher-demand days.
4. **Use a dynamic production policy** — produce based on a rolling forecast.

This is exactly what Monte Carlo simulation helps with: it exposes the consequences of different policies *before* committing to one in real life.

Why Monte Carlo Simulation Matters

Monte Carlo is powerful precisely because most real-world systems don't have neat closed-form mathematical solutions. Some situations where it shines:

Field	Application
Operations / Supply Chain	Inventory planning, demand forecasting
Finance	Stock price simulation, portfolio risk analysis
Engineering	Reliability analysis, structural stress testing
Healthcare	Modelling epidemic spread, surgical scheduling
Project Management	Estimating project completion time under uncertainty
Physics / Nuclear	Neutron diffusion (the original use case)

Real-world example: Before a new petrol station is built, a Monte Carlo simulation might be run thousands of times using historical arrival-rate data to determine how many pump bays

are needed. The average result across thousands of simulated days gives a far more reliable answer than any single calculation.

Key Terms Summary

Term	Meaning
Random Number	A value between 0 and 1 with no predictable pattern; each value equally likely.
Probability Distribution	A table showing how likely each outcome is.
Cumulative Probability	Running total of probabilities — used to define RN ranges.
Random Number Range	The interval of random numbers mapped to one particular outcome.
Simulation Run	One complete pass through the sequence (e.g., one 10-day simulation).
Steady-State	When the simulation runs long enough that the results stabilise and reflect the true long-run behaviour.

Connection to the Rest of the Course

Monte Carlo sits at the intersection of everything covered so far:

- It uses **statistics** (Week 3) to describe the probability distribution of inputs.
- It can simulate **queueing systems** (Week 2) by generating random arrival and service times.
- It applies the **birth and death process** (Week 4) indirectly — random numbers can represent the probability of a birth or death in any given moment.
- It is a core tool in **system simulation** (Week 1) — the "simulation model development" step of the modelling process often means writing a Monte Carlo simulation.

In Python (the language for this course), libraries like `random`, `numpy.random`, and `scipy.stats` make it straightforward to generate random numbers and run Monte Carlo

experiments programmatically — replacing the manual random-number-sequence approach done by hand in these examples.